

Documentation technique — API et base de données

Ce document résume la pile backend, le schéma MongoDB (Prisma), l'authentification, les endpoints HTTP et les règles d'accès par rôle pour le tableau de bord photovoltaïque.

```
**Emplacement du backend : ** `backend/`  
**Schéma Prisma : ** `backend/prisma/schema.prisma`  
**Point d'entrée HTTP : ** `backend/src/index.ts` (préfixe `/api`)
```

1. Pile technique

Couche	Technologie
Runtime	Node.js, TypeScript (ESM)
Framework HTTP	Express
ORM	Prisma (<code>@prisma/client</code>)
Base de données	MongoDB (<code>provider = "mongodb"</code> dans <code>schema.prisma</code>)
Validation des corps de requête	Zod (<code>backend/src/validation/schemas.ts</code>)
Authentification	JWT : access token (court) + refresh token (long), sessions persistées en base
Hachage des mots de passe	bcrypt (via utilitaires du projet)
Import Excel	<code>multer</code> (fichier en mémoire) + <code>xlsx</code>
Export PDF	<code>jspdf</code> , éventuellement fusion avec modèle (<code>pdf-lib</code>) selon la fonction

> **Replica set** : le fichier `.env.example` indique que Prisma MongoDB attend souvent une configuration de type replica set ; le script `scripts/enable-mongo-replicaset.ps1` est mentionné dans les commentaires de configuration.

2. Variables d'environnement (backend)

Principales variables (voir `backend/.env.example`) :

Variable	Rôle
<code>DATABASE_URL</code>	URI MongoDB (base <code>photovoltaïque_dashboard</code> , timeouts possibles)
<code>JWT_ACCESS_SECRET</code> / <code>JWT_REFRESH_SECRET</code>	Clés de signature des JWT
<code>JWT_ACCESS_EXPIRES</code> / <code>JWT_REFRESH_EXPIRES</code>	Durée de vie (ex. <code>15m</code> / <code>7d</code>)
<code>PORT</code>	Port d'écoute (défaut <code>8787</code>)
<code>HOST</code>	Hôte d'écoute (défaut <code>0.0.0.0</code>)
<code>NODE_ENV</code>	<code>development</code> / <code>production</code> (CORS : en prod, <code>FRONTEND_URL</code>)
<code>FRONTEND_URL</code>	Origine autorisée en production pour CORS
<code>SKIP_DEV_ADMIN_BOOTSTRAP</code>	Si <code>1</code> , désactive la création du compte admin de développement au démarrage

Endpoints de santé publics (sans JWT) :

```
- `GET /health` et `GET /api/health` ! `{ ok: true }`
```

3. Schéma de données simplifié (Prisma / MongoDB)

3.1 Énumérations (`backend/prisma/schema.prisma`)

- **Role** : `SUPER\_ADMIN`, `ADMIN`, `TECHNICIEN`, `COMMERCIAL`
- **EtatDossier** : `OUVERT`, `EN\_NEGOCIATION`, `FINIE`, `ARCHIVE`, `ABANDONNE`
- **StatutApprobation** : `APPROUVE`, `PAS\_ENCORE`, `NEANT`, `EN\_ATTENTE`, `REJETE`, `ABANDONNE`
- **TypeContrat** : `PROGRAMME\_PROSOL`, `HORS\_PROGRAMME\_PROSOL`
- **TypeCompteur** : `MONOPHASE`, `TRIPHASE`
- **TypeProjet** : `PHOTOVOLTAIQUE\_CLASSIQUE`, `POMPAGE`, `ISOLE\_AVEC\_BATTERIES`
- **ClassementDossier** : `ARCHIVE`, `NON\_ARCHIVE`

3.2 Modèles et relations

...

User

```
%%% Session[]      (refreshToken, expiresAt)
%%% Projet[]       (relation "AgentCommercial" via agentCommercialId)
%%% ActionLog[]
```

Projet

```
%%% agentCommercialId !' User (optionnel)
%%% champs identité / localisation (référence, abonnés, adresse, GPS, etc.)
%%% champs métier PV / pompage / isolé (puissance, modules, onduleurs, d
%%% contratAchat (TypeContrat), conditionSubvention / saisies / déblocage
%%% Echeance[]      (montants / dates d'échéancier)
%%% ActionLog[]
```

Echeance

```
%%% projetId !' Projet (cascade à la suppression)
```

ActionLog

```
%%% userId !' User
%%% projetId !' Projet (optionnel, SetNull si projet supprimé)
```

...

Les index principaux sur `Projet` couvrent notamment : état + classement, lieu géographique, agent commercial, contrat, références administratives (CIN, code-barres, devis, facture), dates.

3.3 Corps de requête typique pour un projet

La validation Zod `projetBodySchema` (`backend/src/validation/schemas.ts`) définit les champs acceptés pour **création** et **mise à jour** : identité client, type de projet, statuts, dates (souvent en chaînes ISO ou vides), montants (string ou nombre puis normalisation numérique), tableau `echeances`, etc.

Les dates envoyées par le client sont ensuite converties côté service selon la logique métier (fonctions du type `optDate` dans le service projet).

4. Authentification

1. **Login** : `POST /api/auth/login` avec `{ email, password }`.
Réponse typique : `{ accessToken, refreshToken, user }` (utilisateur « public » sans mot de passe).
2. **Requêtes authentifiées** : en-tête
`Authorization: Bearer <accessToken>`
3. **Rafraîchissement** : `POST /api/auth/refresh` avec `{ refreshToken }`.

4. ****Déconnexion**** : ``POST /api/auth/logout`` avec ``{ refreshToken }`` (réponse ****204****).

5. ****Payload JWT access**** : ``sub`` (id utilisateur), ``email``, ``role`` (enum Prisma ``Role``).

5. Table des endpoints API

Tous les chemins ci-dessous sont relatifs à l'origine de l'API (ex. ``http://localhost:8787``).
Sauf mention ****Public****, une authentification par Bearer est requise.

5.1 Authentification — ``/api/auth``

Méthode	Chemin	Auth	Corps / paramètres
POST	<code>`/login`</code>	Public	<code>`{ email, password }`</code>
POST	<code>`/refresh`</code>	Public	<code>`{ refreshToken }`</code>
POST	<code>`/logout`</code>	Public	<code>`{ refreshToken }`</code> ! **204**
GET	<code>`/me`</code>	Bearer	— ! profil utilisateur
PATCH	<code>`/change-password`</code>	Bearer	<code>`{ currentPassword, newPassword }`</code>
GET	<code>`/activity`</code>	Bearer	<code>`?limit`</code> (nombre) — **SUPER_ADMIN** ou **ADMIN** uniquement

5.2 Utilisateurs — ``/api/users``

Méthode	Chemin	Rôles autorisés (middleware)	Description
GET	<code>`/commerciaux`</code>	Tout utilisateur connecté	Liste des commerciaux (usage formulaires, etc.)
GET	<code>`/`</code>	SUPER_ADMIN, ADMIN	Liste des utilisateurs
GET	<code>`/:id`</code>	SUPER_ADMIN, ADMIN	Détail utilisateur
POST	<code>`/`</code>	SUPER_ADMIN, ADMIN	Création (schéma <code>`userCreateSchema`</code> ; pas de création SUPER_ADMIN via ce schéma)
PUT	<code>`/:id`</code>	SUPER_ADMIN, ADMIN	Mise à jour (<code>`userUpdateSchema`</code>)
PATCH	<code>`/:id/toggle`</code>	SUPER_ADMIN, ADMIN	Activer / désactiver
DELETE	<code>`/:id`</code>	**SUPER_ADMIN** seul	Suppression
POST	<code>`/:id/reset-password`</code>	SUPER_ADMIN, ADMIN	Réinitialisation du mot de passe

5.3 Projets — ``/api/projets``

Toutes les routes passent par ``authenticate``.

Méthode	Chemin	Rôles / notes	Description
GET	<code>`/export/excel`</code>	Tous	Export Excel ; filtres via **query** (<code>`ProjetListQuery`</code>) ; sélection par IDs possible via <code>`search=ids:id1,id2`</code>
POST	<code>`/export/excel`</code>	Tous	Même logique + corps JSON : filtres + <code>`columns`</code> (export colonnes sur mesure)
GET	<code>`/export/pdf/columns`</code>	Tous	Métadonnées des colonnes PDF (liste des clés et libellés)
GET	<code>`/export/pdf`</code>	Tous	Export PDF liste (query = filtres + colonnes)
POST	<code>`/export/pdf`</code>	Tous	Idem avec filtres/colonnes dans le **corps** (URLs longues)
POST	<code>`/import`</code>	**SUPER_ADMIN, ADMIN, TECHNICIEN**	Upload <code>`multipart/form-data`</code> , champ fichier <code>`file`</code>
POST	<code>`/archive`</code>	Tous	<code>`{ ids: string[], archive: boolean }`</code>
GET	<code>`/`</code>	Tous	Liste paginée + filtres (query : voir § 5.5)
POST	<code>`/`</code>	Tous	Création — corps <code>`projetBodySchema`</code> normalisé
GET	<code>`/:id/export/pdf`</code>	Tous	PDF fiche projet unique
GET	<code>`/:id`</code>	Tous	Détail projet

| PUT | `/:id` | Tous | Mise à jour — même schéma que la création |
| DELETE | `/:id` | ****SUPER_ADMIN**** ou ****ADMIN**** | Suppression |

****Service métier**** : ****l'import Excel refuse explicitement le rôle **COMMERCIAL** (`403`), en plus du middleware de route.**

5.4 Statistiques — `/api/stats`

Toutes les routes : ****Bearer**** requis ; pas de `authorize` dans `stats.routes.ts` (tout rôle connecté).

GET	Chemin
`/dashboard`	
`/par-district`	
`/par-etat`	
`/par-contrat-achat`	
`/par-mois`	
`/financier`	
`/prosol-par-district`	
`/par-deblocage`	
`/recoulement-par-district`	
`/diligences-par-district`	

Les paramètres exacts dépendent des contrôleurs `stats.controller.ts` (filtres éventuels en query).

5.5 Filtres de liste projet (query `ProjetListQuery`)

Utilisés notamment par `GET /api/projets` et les exports. Champs principaux (tous optionnels, souvent des chaînes) :

pagination : `page`, `limit` ; recherche : `search` ; filtres : `etat`, `district`, `contrat`, `typeProjet`, `banque`, `agentId`, `classement`, champs géographiques, approbations, compteur, références, marques/modèles, montants min/max, `tauxInteret`, `lotDeblocage`, plage de dates `dateFrom` / `dateTo` / `dateField`, tri : `sortBy`, `order`, etc.

(détail complet dans `backend/src/services/projet.service.ts`, type `ProjetListQuery`.)

6. Règles par rôle (récapitulatif)

Rôle	Accès API typique
****SUPER_ADMIN****	Accès complet utilisateurs (y compris suppression) ; routes admin auth (`/activity`) ; toutes routes projets y compris suppression ; import Excel.
****ADMIN****	Comme SUPER_ADMIN sauf ****DELETE**** utilisateur (réservé SUPER_ADMIN) ; `/activity` ok.
****TECHNICIEN****	Pas de gestion des utilisateurs (liste/détail/CRUD) ; import Excel autorisé ; projets : pas de DELETE projet via route (réservé ADMIN/SUPER_ADMIN).
****COMMERCIAL****	Utilisateurs : seulement `GET /api/users/commerciaux` ; pas d'import Excel (403 dans le service) ; pas de suppression de projet.

6.1 Point d'attention : filtrage « commercial » côté backend

Dans `projet.service.ts`, la fonction `commercialFilter` retourne actuellement ****`null`**** (aucun filtre supplémentaire pour les commerciaux).

Dans `stats.service.ts`, `commercialWhere` retourne un objet ****`{}`**** (pas de restriction par `agentCommercialId`).

Conséquence : ****un utilisateur COMMERCIAL authentifié peut, côté API, lister / lire / modifier des projets comme les autres rôles non restreints par middleware**** — la limitation stricte à « ses seuls dossiers » n'est pas appliquée dans ces services. Le tableau de bord frontend peut masquer des sections (UX), mais la sécurité serveur doit être alignée si l'exigence métier est de isoler les données par commercial.

7. Fichiers utiles pour la maintenance

Sujet	Fichiers
-----	-----
Routes	`backend/src/routes/*.routes.ts`
Contrôleurs	`backend/src/controllers/*.controller.ts`
Logique métier projets / exports	`backend/src/services/projet.service.ts`
Statistiques	`backend/src/services/stats.service.ts`
Auth	`backend/src/services/auth.service.ts`, `middleware/auth.middleware.ts`, `middleware/role.middleware.ts`
JWT	`backend/src/utls/jwt.ts`
Validation Zod	`backend/src/validation/schemas.ts`

Document généré à partir du code du dépôt ; en cas de divergence, le code source fait foi.